

The traps of asynchronous computing

```
<!DOCTYPE html>
<head>
  <title>AJAJ Demo</title>
  <script>...</script>
</head>
<body>
  <form>
    <input type= "button" onclick=
      'document.getElementById("myPar").innerHTML=getJson();',
      value= "Get it!">
    </form>
    <p id="myPar">here the json will appear</p>
  </body>
</html>
```

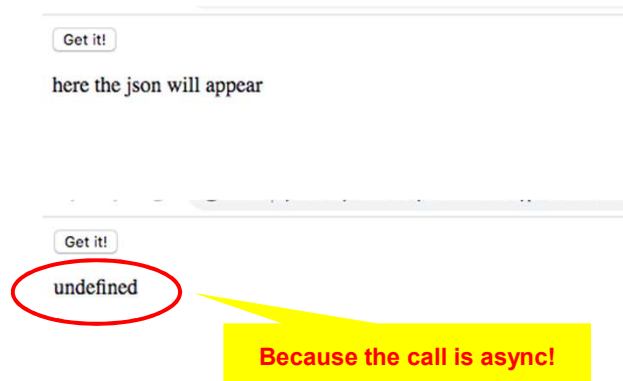
27

The traps of asynchronous computing

```
<script>
function getJson() {
  var my_JSON_object;
  var url= "https://mdn.github.io/learning-
    area/javascript/ojs/json/superheroes.json"
  var xhttp = new XMLHttpRequest();
  xhttp.responseType = "json";
  xhttp.onreadystatechange = function () {
    if (this.readyState == 4){
      if (this.status == 200)
        { my_JSON_object = this.response;
          console.log("Dati recuperati"); }
        else {
          console.log("Errore di caricamento: " + this.status); }
      }
    };
  xhttp.open("GET", url, true);
  xhttp.send();
  return my_JSON_object;
}
</script>
```

28

The traps of asynchronous computing 1



29

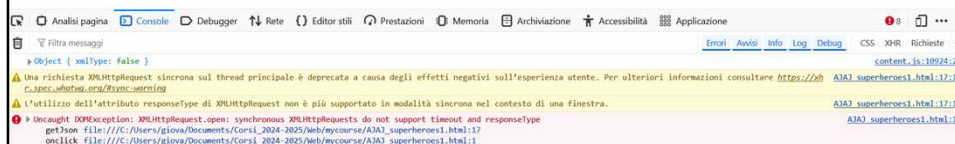
The traps of asynchronous computing

```
<script>
function getJson() {
  var my_JSON_object;
  var url= "https://mdn.github.io/learning-
    area/javascript/ojs/json/superheroes.json"
  var xhttp = new XMLHttpRequest();
  xhttp.responseType = "json";
  xhttp.onreadystatechange = function () {
    if (this.readyState == 4){
      if (this.status == 200)
        { my_JSON_object = this.response;
          console.log("Dati recuperati"); }
        else {
          console.log("Errore di caricamento: " + this.status); }
    }
  };
  xhttp.open("GET", url, false);
  xhttp.send();
  return my_JSON_object;
}
</script>
```

Let us make it sync

30

The traps of asynchronous computing



31

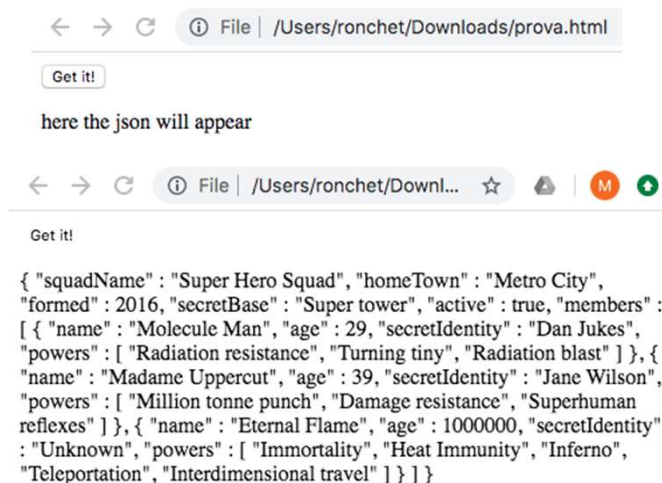
The traps of asynchronous computing

```
<script>
function getJson() {
  var my_JSON_object;
  var url= "https://mdn.github.io/learning-
    area/javascript/ojs/json/superheroes.json"
  var xhttp = new XMLHttpRequest();
  //xhttp.responseType = "json";
  xhttp.onreadystatechange = function () {
    if (this.readyState == 4){
      if (this.status == 200)
        { my_JSON_object = this.response;
          console.log("Dati recuperati"); }
        else {
          console.log("Errore di caricamento: " + this.status); }
        }
    };
  xhttp.open("GET", url, false);
  xhttp.send();
  return my_JSON_object;
}
</script>
```

Let's comment this line

32

The traps of asynchronous computing



33

The traps of asynchronous computing

```
<!DOCTYPE html>
<head>
  <title>AJAJ Demo</title>
  <script>...</script>
</head>
<body>
  <form>
    <input type= "button" onClick='getJson();'>
  </form>
  <p id="myPar">here the json will appear</p>
</body>
</html>
```

**This is the right way
of doing things!**

34

The traps of asynchronous computing

```

<script>
function getJson() {
  var my_JSON_object;
  var url= "https://mdn.github.io/learning-
    area/javascript/ojs/json/superheroes.json"
  var xhttp = new XMLHttpRequest();
  xhttp.responseType = "json";
  xhttp.onreadystatechange = function () {
    if (this.readyState == 4){
      if (this.status == 200)
      { my_JSON_object = this.response;
        console.log("Dati recuperati");
        document.getElementById("myPar").innerHTML =
          my_JSON_object.squadName; }
      else {
        console.log("Errore di caricamento: " + this.status); }
    }
  };
  xhttp.open("GET", url, true);
  xhttp.send();
  //return my_JSON_object;
35 }
</script>

```

This is the right way
of doing things!

Let's make it async again

The traps of asynchronous computing

Get it!

here the json will appear

Get it!

Super Hero Squad

36

The traps of asynchronous computing

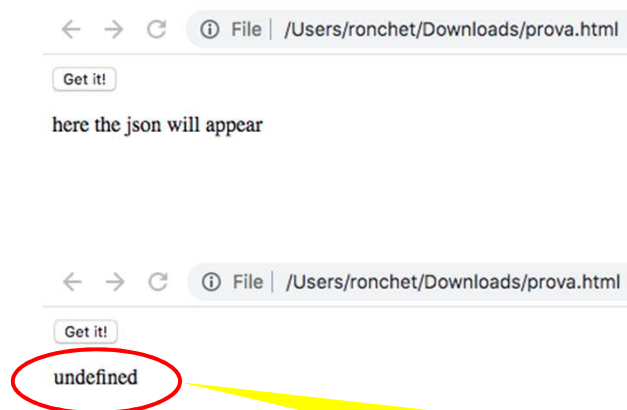
```

<script>
function getJson() {
  var my_JSON_object;
  var url= "https://mdn.github.io/learning-
    area/javascript/ojs/json/superheroes.json"
  var xhttp = new XMLHttpRequest();
  xhttp.responseType = "json";
  xhttp.onreadystatechange = function () {
    if (this.readyState == 4){
      if (this.status == 200)
        { my_JSON_object = this.response;
          console.log("Dati recuperati");
          document.getElementById("myPar").innerHTML =
            my_JSON_object.squadName; }
        else {
          console.log("Errore di caricamento: " + this.status); }
      }
    };
    xhttp.open("GET", url, true);
    xhttp.send();
  }
  /script>

```

Let's comment this line

The traps of asynchronous computing



The JavaScript JSON object

- The built-in global **JSON** object contains the following two methods: `parse()` and `stringify()`
- Let's have a look at the methods `parse`
 - **`JSON.parse(text[, reviver])`** Parse the string *text* as JSON, optionally transform the produced value and its properties, and return the value. Any violations of the JSON syntax, including those pertaining to the differences between JavaScript and JSON, cause a `SyntaxError` to be thrown.

39

JSON.parse: from a JSON string to a JS object

```
const json = '{"result":true, "count":42}';  
const obj = JSON.parse(json);  
  
console.log(obj.count);    // expected output: 42  
  
console.log(obj.result);   // expected output: true
```

40

JSON.parse with reviver function 1

```
var text = '{ "name":"Dorothea Wierer",
              "birthDate":"1990-04-03",
              "city":"Brunico"}';

var obj = JSON.parse(text, function (key, value) {
    if (key == "birthDate") {
        return new Date(value);
    } else {
        return value;
    }
});
```



reviver function

41

JSON.parse with reviver function 2

```
var text = '{ "name":"Dorothea Wierer",
              "birthDate":"1990-04-03",
              "city":"Brunico"}';

function rev(key,value){
    if (key == "birthDate") {
        return new Date(value);
    } else {
        return value;
    }
}

var obj = JSON.parse(text,rev);
```

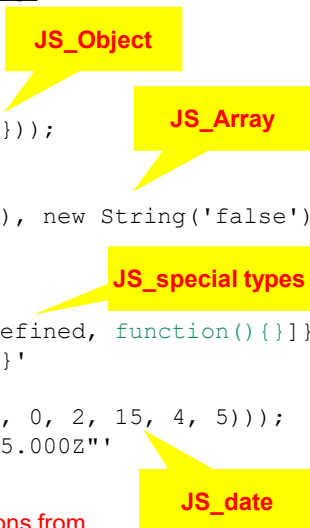
42

The JavaScript JSON object

- The built-in global **JSON** object contains the following two static methods: `parse()` and `stringify()`
- Let's have a look at the method `stringify()`:
 - **`JSON.stringify(value [, replacer])`**. Convert a JS value to a JSON string, optionally replacing values if a replacer function is specified or optionally including only the specified properties if a replacer array is specified.

43

JSON.stringify



```

console.log(JSON.stringify({ x: 5, y: 6 }));
// expected output: '{"x":5,"y":6}'

console.log(JSON.stringify([new Number(3), new String('false'),
new Boolean(false)]));
// expected output: '[3,"false",false]'

console.log(JSON.stringify({ x: [10, undefined, function(){}]}));
// expected output: '{"x":[10,null,null]}'

console.log(JSON.stringify(new Date(2006, 0, 2, 15, 4, 5)));
// expected output: '"2006-01-02T15:04:05.000Z"'
  
```

The `JSON.stringify()` function will remove any functions from a JavaScript object, both the key and the value

44

JSON.stringify

- Using a function as replacer

```
function replacer(key, value) {
  // Filtering out properties
  if (typeof value === "string") {
    return undefined;
  }
  return value;
}

const foo = {
  foundation: "Mozilla",
  model: "box",
  week: 45,
  transport: "car",
  month: 7,
};
JSON.stringify(foo, replacer);

'{"week":45,"month":7}'
```

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON/stringify

45

JSON.stringify

- Using an array as replacer

```
const foo = {
  foundation: "Mozilla",
  model: "box",
  week: 45,
  transport: "car",
  month: 7,
};

JSON.stringify(foo, ["week",
"month"]);

'{"week":45,"month":7}'
```

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON/stringify

46

What is the most modern way to write AJAX/AJAJ requests?

47

JS Promises: a short introduction

- Let's define:
 - *Producing code*: code that does something and takes time to do it (e.g. code for loading a resource over a network)
 - *Consuming code*: code that wants obtain the results of the producing code once they are available
- A promise is a special JS object linking the producing code to the consuming code, making the results of the producing code **available when they are ready**

```
let promise = new Promise(function(resolve, reject) {  
  // executor});
```

A very good tutorial: <https://javascript.info/promise-basics>

48

JS Promises: the executor function

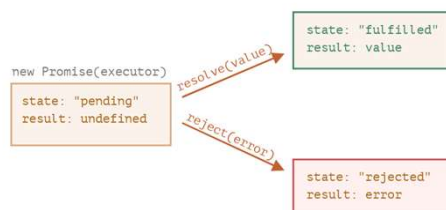
- The function passed to new Promise is called **executor** that contains the *producing code* expected to provide results. So the executor is the producer.
- Executor is automatically run when the new promise is created and tries to complete its job
- Once the executor finished its job, it calls a JS call back:
 - **resolve** (value) – if the job is finished successfully
 - **reject** (error) – if an error occurred

A very good tutorial: <https://javascript.info/promise-basics>

49

JS Promises: the promise properties

- The created promise has 2 properties:
 - **State**: initially **pending** it can changes to either **fulfilled** when **resolve** is called or **rejected** when **reject** is called
 - **Result**: initially **undefined**, then changes to **value** when **resolve(value)** is called or **error** when **reject(error)** is called



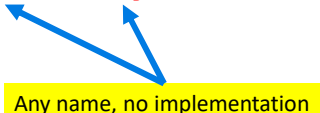
Picture from <https://javascript.info/promise-basics>

50

JS Promises

- Creating the Promise and evaluating it:

```
let promise = new Promise(function(resolve, reject) {
  ... some code...
  if (ok) {
    resolve(valueToBeReturned);
  } else {
    reject(new Error("message"));
  }
})
```



Any name, no implementation

51

JS Promises: caveats

- There can be only a single result or an error
 - The executor should call only one resolve or one reject
 - Any state change is final

```
let promise = new Promise(function(resolve, reject) {
  resolve("done");

  reject(new Error("...")); // ignored
  setTimeout(() => resolve("...")); // ignored
});
```

- Reject with Error object
 - It is recommended to use Error object as parameters of reject

52

JS Promises: caveats

- Immediately calling resolve / reject

- An executor usually does something asynchronously and calls resolve/reject after some time. It also can call resolve or reject immediately

```
let promise = new Promise(function(resolve, reject) {
  // not taking our time to do the job
  resolve(123); // immediately give the result: 123
})
```

- The state and result are not directly accessible

- State and result are accessible through methods, e.g. then

53

JS Promises: consumers

- The most important consumer is then

```
promise.then(
  function(value) {doSomethingWithValue(value);}, //if fulfilled
  function(error) {doSomethingWithError(error);} //if rejected
);
```

- If only successful operations are interesting, you can use then with only one function argument

- If only errors are interesting, you can use:

- then putting null as first function argument


```
then(null, errorHandlingFunction)
```
- catch(errorHandlingFunction)

54

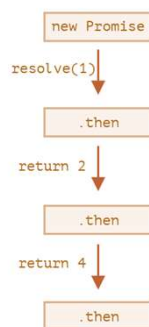
Promises chaining

- `.then` returns a newly generated promise, that can eventually be use for chaining

```

1 new Promise(function(resolve, reject) {
2
3   setTimeout(() => resolve(1), 1000); // (*)
4
5 }).then(function(result) { // (**)
6
7   alert(result); // 1
8   return result * 2;
9
10 }).then(function(result) { // (***)
11
12   alert(result); // 2
13   return result * 2;
14
15 }).then(function(result) {
16
17   alert(result); // 4
18   return result * 2;
19
20 });

```



Picture from <https://javascript.info/promise-basics>

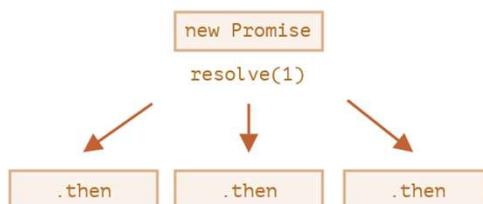
55

Promises with multiple consumers

```

1 let promise = new Promise(function(resolve, reject) {
2   setTimeout(() => resolve(1), 1000);
3 });
4
5 promise.then(function(result) {
6   alert(result); // 1
7   return result * 2;
8 });
9
10 promise.then(function(result) {
11   alert(result); // 1
12   return result * 2;
13 });
14
15 promise.then(function(result) {
16   alert(result); // 1
17   return result * 2;
18 });

```



Picture from <https://javascript.info/promise-basics>

56

Let's try promises! ...with HTTP

`promises.html`

57

Find the differences!

- Let's analyze the code at the slide 37 and that one in the file `promises.html`
- The output is exactly the same thing
- Do they behave in the same way?

58

Another way to do

59

JS fetch

- The fetch method is used to load information from remote servers

```
let promise = fetch(url)
```

- This makes a network request to the url and returns a promise. The promise resolves with a response object when the remote server responds with headers, but before the full response is downloaded
- To read the full response, we should call the method `response.text()` : it returns a promise that resolves when the full text is downloaded from the remote server, with that text as a result
- The response object returned from fetch also includes the method `response.json()` that reads the remote data and parses it as JSON

60

A toy example

```
fetch('./paperino.json')
  .then(function(response) {return response.json();},
    function(error) {console.error("Network error:", error);})
  .then(function(data) {console.log(data.x.y.notexisting);},
    function(error) {console.error("Processing error:", error);});
```

61

More info about .catch

- You can use it to avoid to write the second function parameter for each `.then`, and you want to handle all the errors in the same place, that is at the end of the chain
- You can catch errors using `catch`

62

Ok, let's try it!

```
fetch('./paperino.json')
  .then(function(response) {return response.json(); })
  .then(function(data) {console.log(data.x.y.notexisting);})
  .catch(function(error) {console.error("Any kind of error:", error);});
```

63

More info about .catch

- What is caught by catch?
 - Network errors (e.g. no connection)
 - Thrown errors
 - HTTP errors MUST be manually detected and thrown!**
 - Parsing errors (es. response.json() on a not valid file)

64

Let's try fetch!

fetch.html

65

More info about .catch

- HTTP errors **MUST** be manually detected and thrown!
- We can use the property `ok` of response that takes all the HTTP errors into account
- `response.ok` returns a boolean value:
 - true if the status code is in the range [200,299]
 - false if the status code is in > 400
- If the status code is 3xx, fetch does automatically a new request and return another code

66

JS fetch: summary

- fetch takes a HTTP Request object (may have headers)
- fetch returns a Promise that resolve to a Response object
- Response has methods:
 - `text()` returns a Promise that resolves to a string
 - `json()` returns a Promise that resolves to a Json object
 - `blob()` returns a Promise that resolves to a Blob object

67

Can we do better than .then()?

- Every `.then()` forces us to lock our code inside a new function, breaking the natural flow of reading
- The downloaded data only exists inside that specific `.then()` block, making it hard to share or use elsewhere
- The code gets buried under endless functions and brackets, making it messy and hard to track

68

async and await

- To solve the problem, JavaScript introduced syntactic sugar:
 - `async`: it is a prefix for functions so that they always return a promise, regardless of what explicitly returned in the code. The use of `async` unlocks the use of `await` inside the function.
 - `await`: it is an operator that can only be placed before a promise. It temporarily pauses the execution of that specific `async` function. It unwraps the promise once it settles
 - `try...catch`: It replaces `.catch`. It is typical of synchronous programming

69

Let's try async-await!

[async_await.html](#)

70